Autômatos e Linguagens

Autômatos Finitos

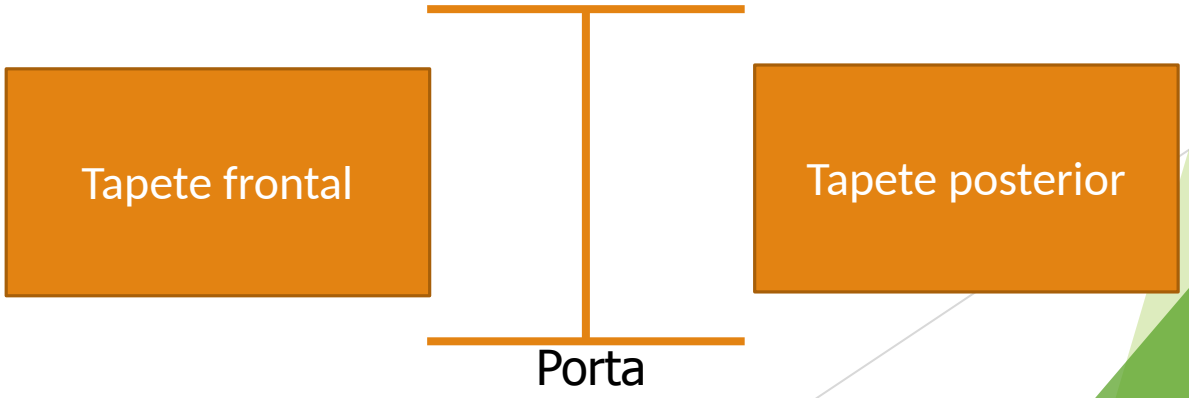
- São modelos para computadores com uma quantidade limitada de memória
- Exemplo:
  - As lavadoras de louça/roupa, termômetros eletrônicos, relógios digitais, calculadoras e máquinas de venda automática...
- Os autômatos também são chamados de **máquinas de estados finitos**

Autômatos Finitos: exemplo Controlador para uma porta automática de entrada

Sinal de entrada

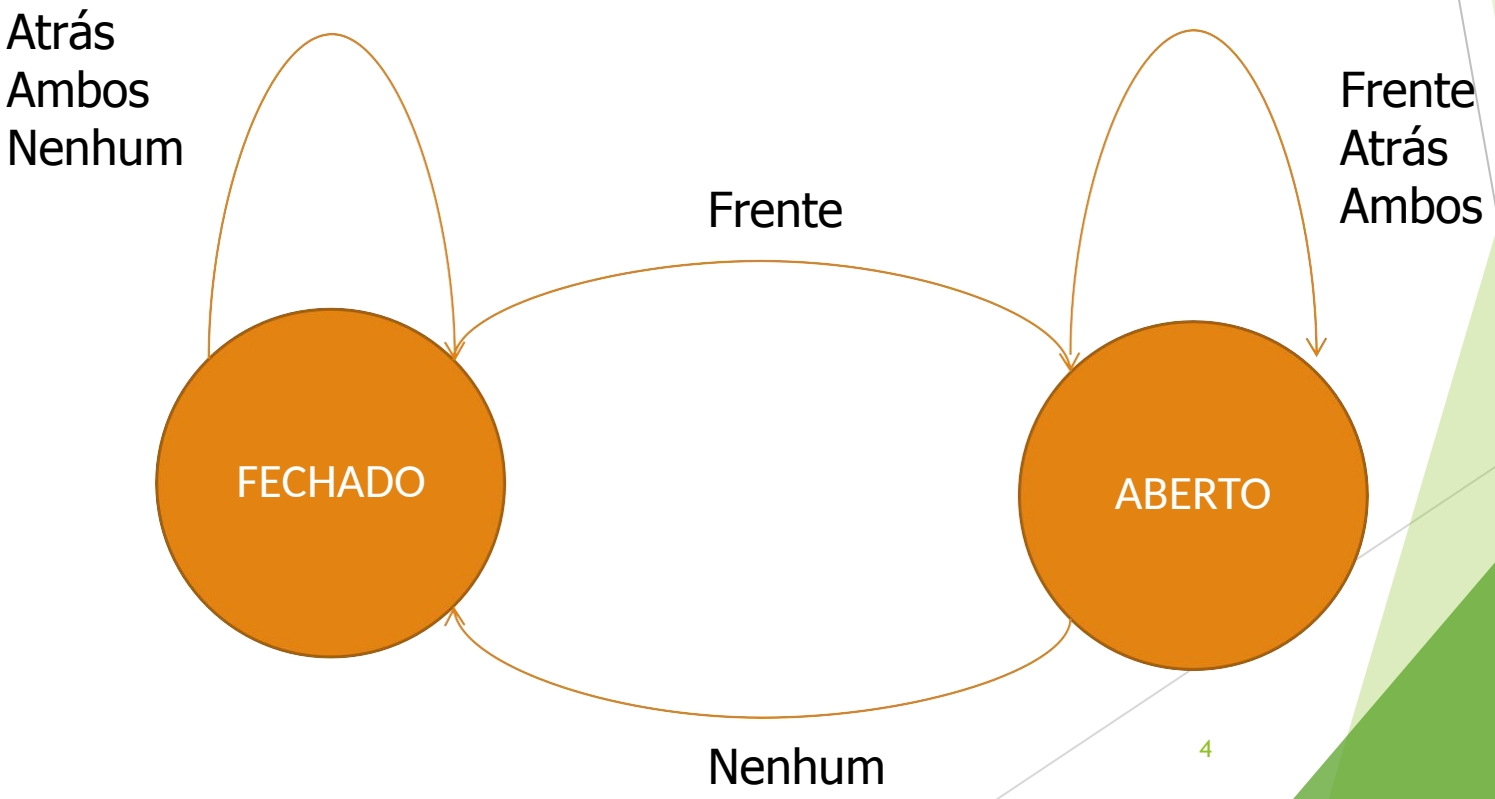
Estado	Nenhum	Frente	Atrás	Ambos
Fechado	Fechado	Aberto	Fechado	Fechado
Aberto	Fechado	Aberto	Aberto	Aberto

Esse controlador é um computador com apenas 1 bit de memória, capaz de registrar em quais dos dois estados o controlador está.



Autômatos Finitos: exemplo Controlador para uma porta automática de entrada

- O controlador se move de estado para estado, dependendo da entrada que ele recebe.



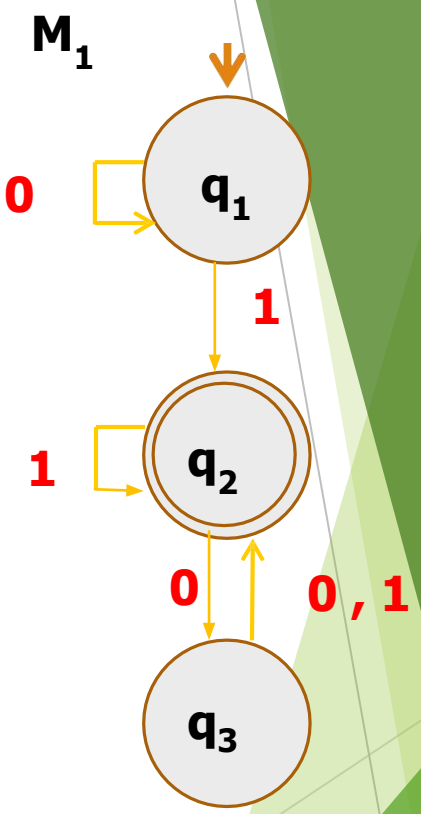
Autômatos Finitos

- Ao começar a descrever a teoria matemática de autômatos finitos, fazemos isso no nível abstrato, sem referência a qualquer aplicação específica.

- A seguir vamos ver alguns exemplos usando um diagrama de estados e identificar os conceitos de: estado inicial, estado de aceitação ou final, transição.

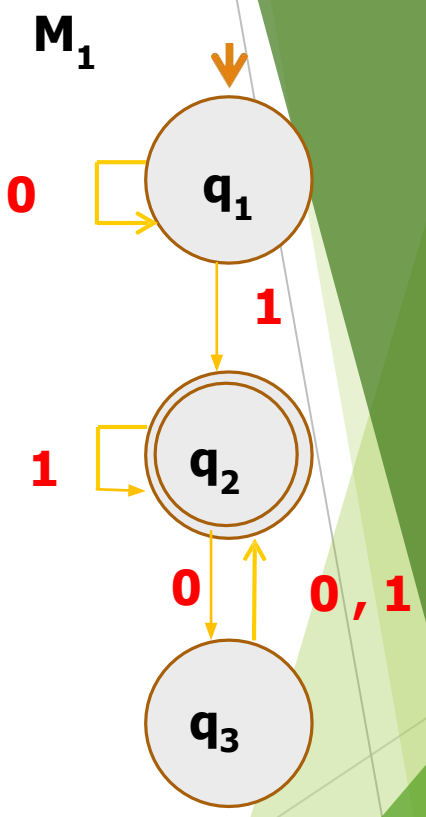
Autômatos Finitos

- **Diagrama de estado** de M1
- **Três estados**: q1, q2 e q3
- **O estado inicial**: q1
  - É indicado pela seta apontando para ele a partir do nada
- **Estado de Aceitação:** q2
  - É aquele com o círculo duplo
- **Transições:** Setas saindo de um estado para o outro



Autômatos Finitos

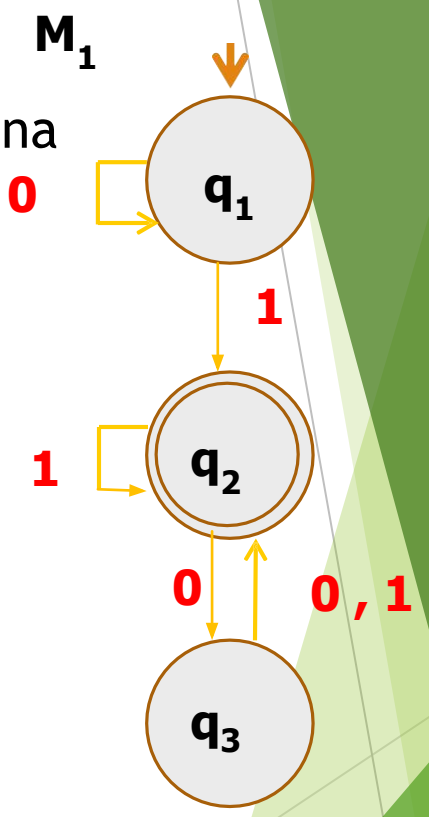
- O autômato recebe os símbolos da cadeia de entrada um por um da esquerda para a direita.
- Após ler cada símbolo, M1 move de um estado para outro ao longo da transição que tem aquele símbolo como seu rótulo.
- Quando ele lê o último símbolo, M1 produz sua saída.
- A saída é aceita se M1 está agora no estado de aceitação e rejeitada se ele não está.



Autômatos Finitos

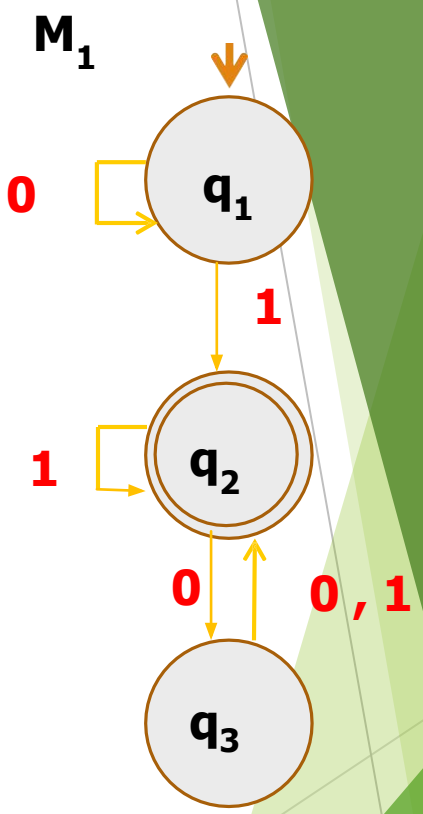
- Quando alimentamos a cadeia 1101 à máquina M1, o processamento procede da seguinte forma:

1. Começa no estado q1
2. Lê 1, segue a transição q1 para q2
3. Lê 1, segue a transição q2 para q2
4. Lê 0, segue a transição q2 para q3
5. Lê 1, segue a transição de q3 para q2
6. Aceita porque M1 está no estado de aceitação q2 no final da entrada



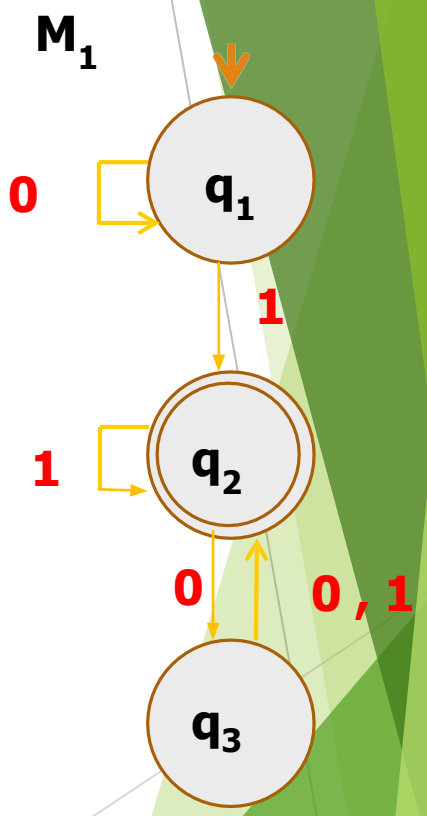
Autômatos Finitos

- Alimentando a Máquina M1 com uma variedade de cadeias
- Percebemos que a máquina aceita as sequencias 1, 01, 11 e 0101010101
- A máquina M1 aceita qualquer cadeia que termine com 1



Autômatos Finitos

- Definição Formal: é uma 5-upla M = (Σ, Q , δ , q 0, F)
  - Σ: alfabeto, símbolos de entrada
  - Q : conjunto finito de estados possíveis
  - δ : função programa de transição
    - $\delta : Q \times \Sigma \rightarrow Q$
  - q0 : estado inicial
  - F: conjunto finito de estados finais

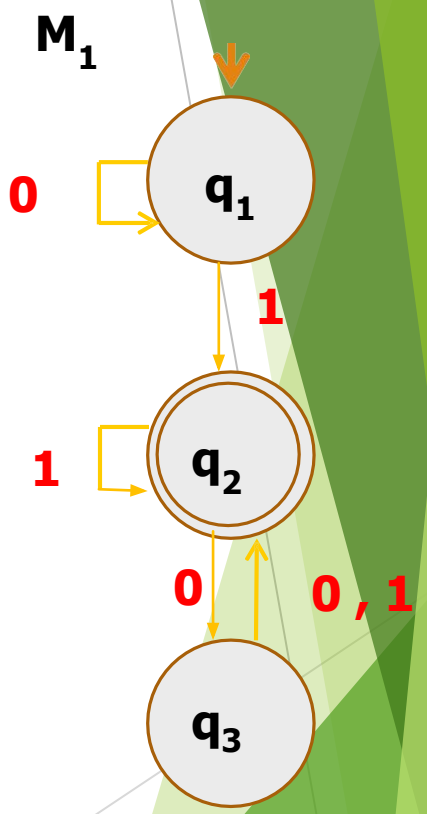


Autômatos Finitos

- Definição Formal: é uma 5-upla M = (Σ, Q , δ , q 0, F)
  - Σ: {0,1}
  - Q: { q_1, q_2, q_3}
  - δ

	0	1
q1	q1	q2
q2	q3	q2
q3	q2	q2

  - q0 : q1
  - F = {q2}



Linguagens

- Se **A** é o conjunto de cadeias que a máquina **M** aceita, dizemos que **A é a linguagem da máquina M** e escrevemos **L(M) = A**
- M reconhece A
  - A máquina M reconhece a linguagem A
- A = { w| w contém pelo menos um 1}

- Uma linguagem é chamada de **linguagem regular** quando um autômato finito a reconhece.

Projetando Autômatos Finitos

- Suponha que lhe é dada alguma linguagem e você deseja projetar um AF que a reconheça.
- Faça de conta que você é o autômato. Você recebe uma cadeia de entrada e tem que determinar se ela é um membro da linguagem que o AF deve reconhecer.
- Você vai vendo os símbolos(possibilidades) na cadeia um por um. Depois de cada símbolo você tem que decidir se a cadeia vista até então está na linguagem ou não.

Projetando Autômatos Finitos

- Você tem que perceber o que precisa lembrar sobre a cadeia na medida em que a está lendo.
- Por que não simplesmente lembrar de tudo que você viu?
  - Lembre-se: Autômatos finitos têm um número limitado de estados.
  - Felizmente, para muitas linguagens você não precisa lembrar de toda a entrada.

Projetando Autômatos Finitos: Exemplo

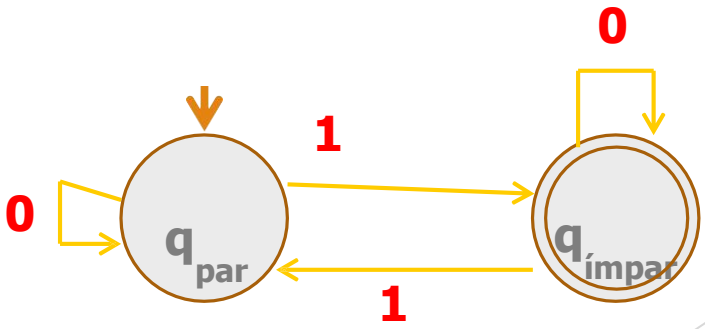
- Suponha que o alfabeto seja {0,1} e que a linguagem consista de todas as cadeias com um número ímpar de 1s.
- Faça de conta que você é o autômato E1, que reconhece essa linguagem. Você recebe uma cadeia de entrada de 0s e 1s.
- Você precisa lembrar a cadeia inteira vista até então para determinar se o número de 1s é ímpar?

Projetando Autômatos Finitos: Exemplo

- Basta lembrar se o número de 1s visto até então é par ou ímpar e manter essa informação à medida que lê novos símbolos.
- Você representa essa informação como uma lista finita de possibilidades:
  - par até agora, e
  - ímpar até agora.
- Então temos um estado para cada possibilidade

Projetando Autômatos Finitos: Exemplo

- Em seguida, você atribui as transições vendo como ir de uma possibilidade para a outra ao ler um símbolo.
- Em seguida, você coloca como sendo estado inicial aquele correspondendo à possibilidade associada como ter visto 0 símbolos até então.
- Por último, ponha como estados de aceitação aqueles correspondendo a possibilidade nas quais você deseja aceitar a cadeia de entrada.



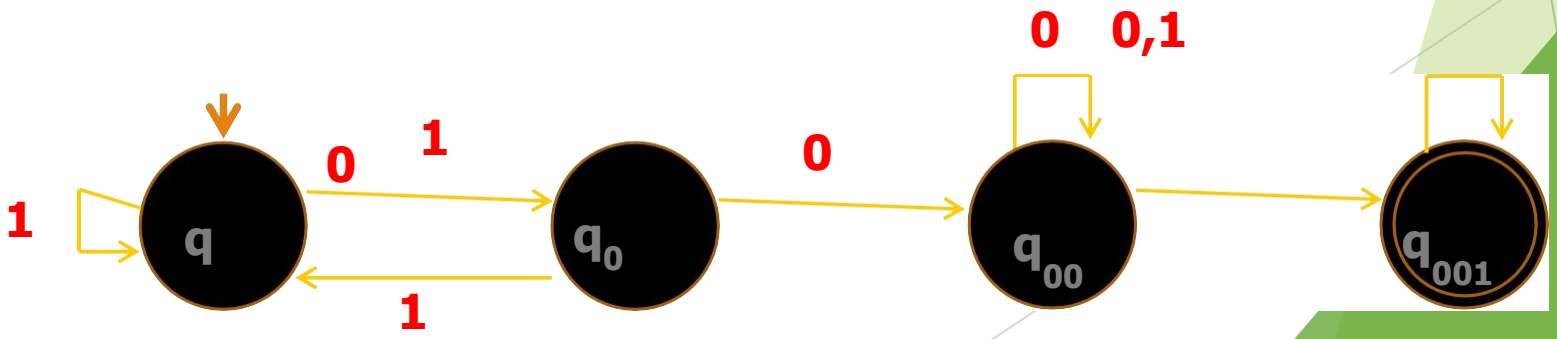
Projetando Autômatos Finitos: Outro exemplo

- Para reconhecer a linguagem de todas as cadeias binárias que contem 001 como uma subcadeia.
- Como você reconheceria essa linguagem?
  - Você inicialmente saltaria sobre todos os 1s.
  - Se você chegar num 0, então você pode ter acabado de ver o primeiro dos três símbolos no padrão 001
  - Se nesse ponto você vê um 1, houve muito poucos 0s, portanto você volta a saltar sobre 1s. Mas, se você vê um 0 nesse ponto, você deve lembrar que você acabou de ver dois símbolos do padrão.
  - Agora você precisa encontrar um 1.
  - Se você o encontrar, logo você conseguiu achar o padrão. Continue lendo a cadeia de entrada até que você chegue no final.

Projetando Autômatos Finitos: Outro exemplo

- Portanto, existem 4 possibilidades:

1. não tem visto quaisquer símbolos do padrão,
2. acaba de ver um 0,
3. acaba de ver 00, ou
4. acaba de ver o padrão inteiro 001.



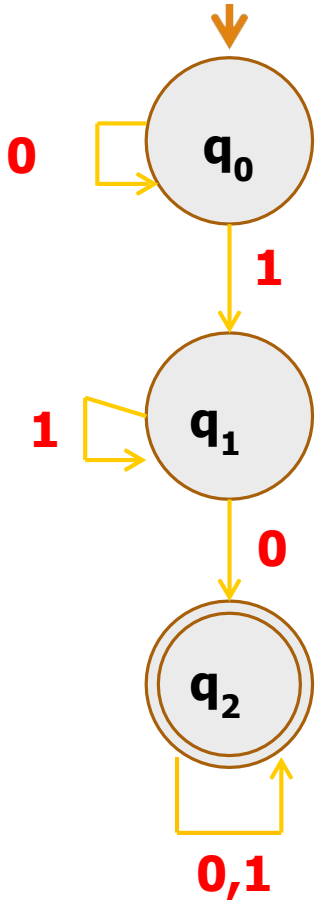
Autômatos Finitos

- Exercícios:

1. Construa um AF M2 que recebe cadeias de bits e aceita aquelas que possuem 10 como subcadeia.
2. Construa um AF M3 que recebe uma cadeia de bits e aceita apenas aquelas que terminam em 1.
3. Construa um AF M4 que recebe uma cadeia de bits podendo ser uma cadeia vazia ε ou termina em 0
4. Construa um AF M5 que possui um alfabeto {a,b} e sempre termina com o símbolo que começou.
5. Construa um AF M6 que aceita as cadeias sobre o alfabeto {a,b} e que possuem aa ou bb como subcadeias.
6. Construa um AF M7 que possui o alfabeto {Reset, 0, 1, 2} e aceita cadeias onde a soma de seus símbolos é múltiplo de 3. E o reset zera a contagem a partir dele.

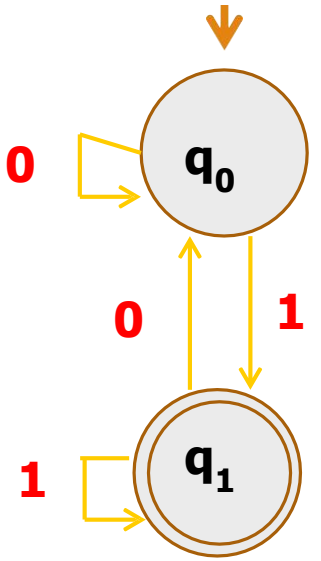
Autômatos Finitos

M_2



Autômatos Finitos

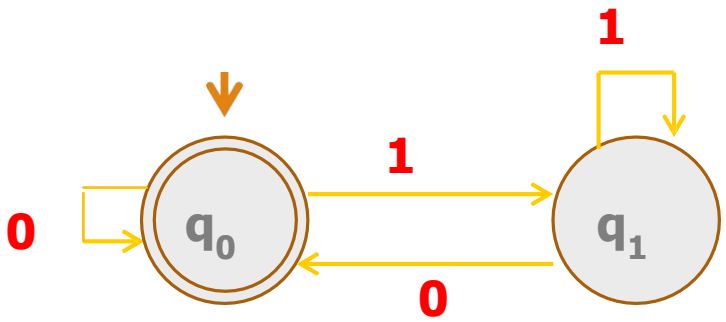
M_3



L(M3) = { w | w termina com um 1}

Autômatos Finitos

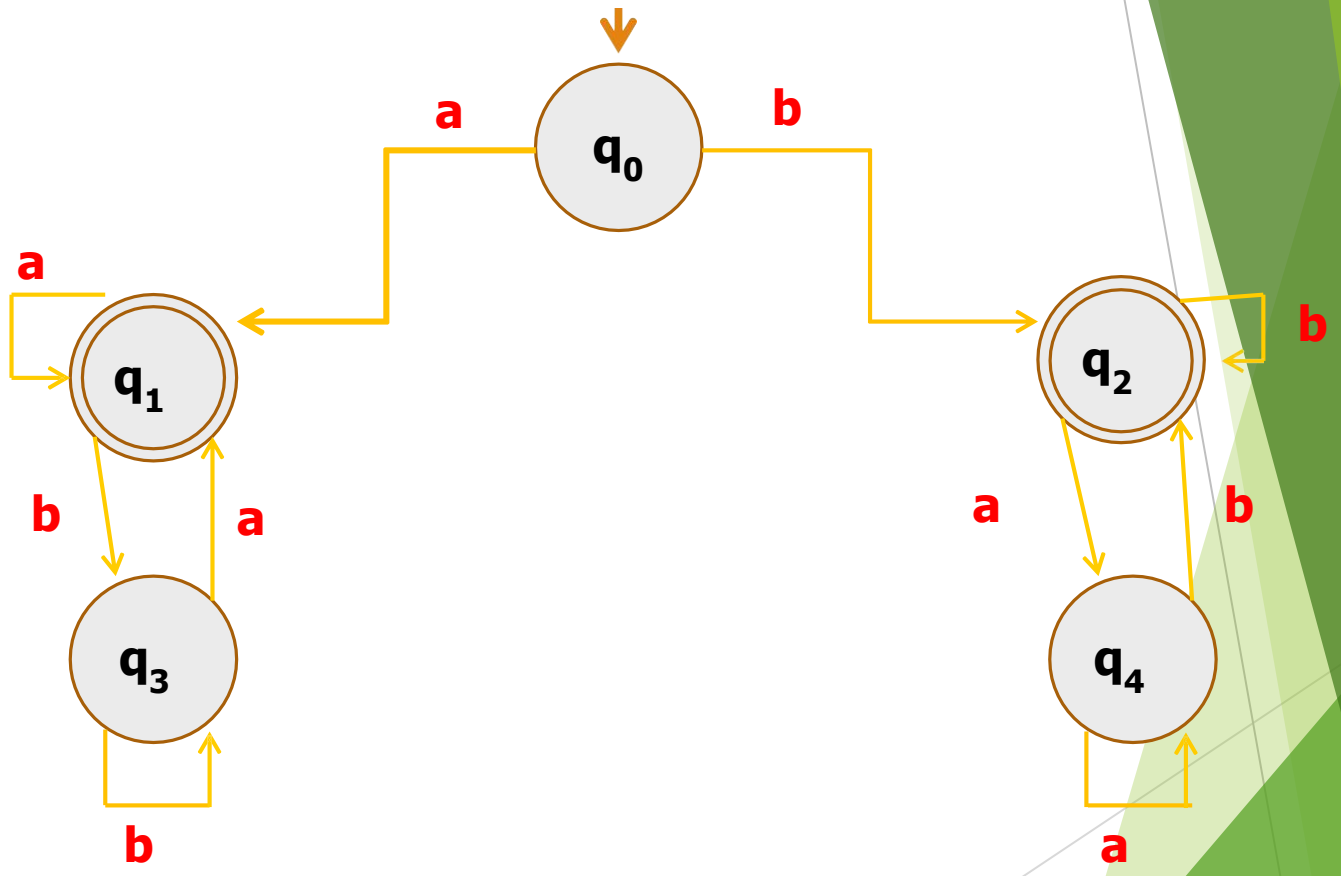
M_4



L(M4) = { w | w é a cadeira vazia ε ou termina com um 0}

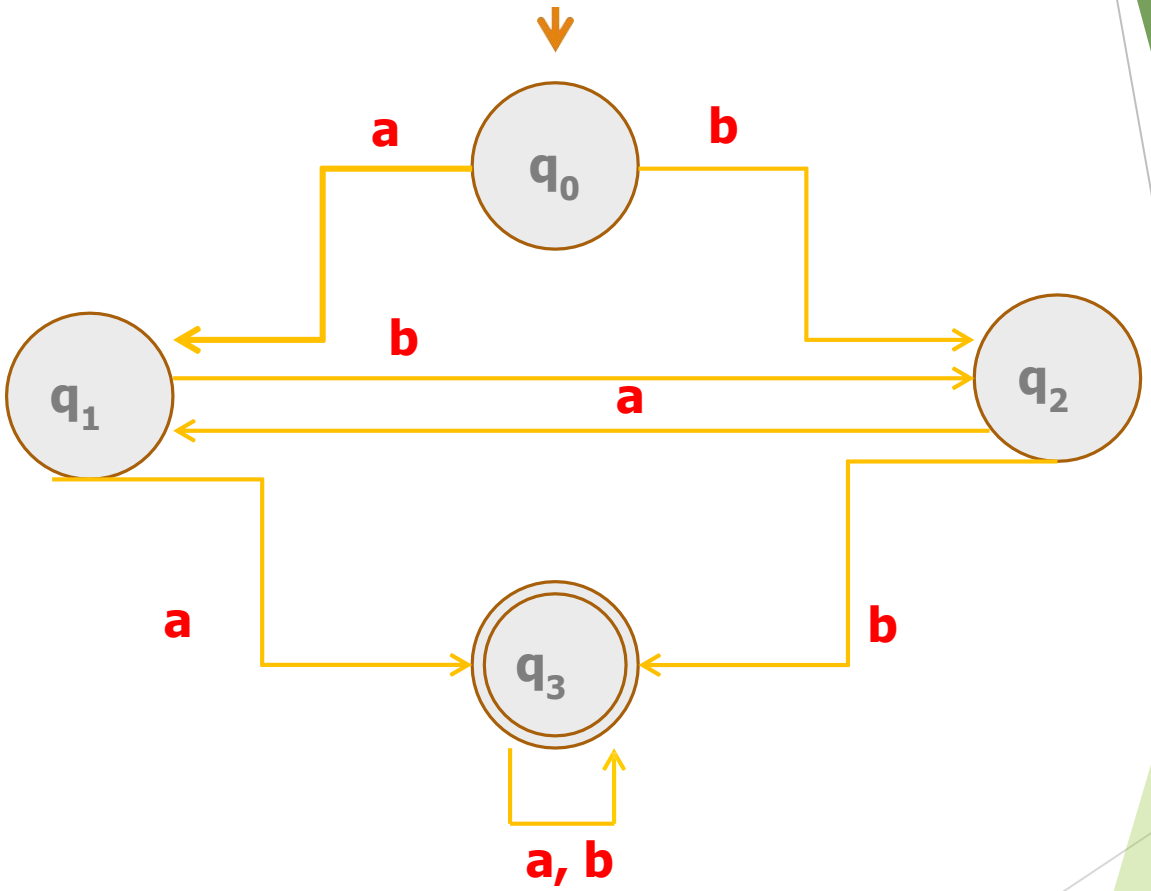
Autômatos Finitos

M_5



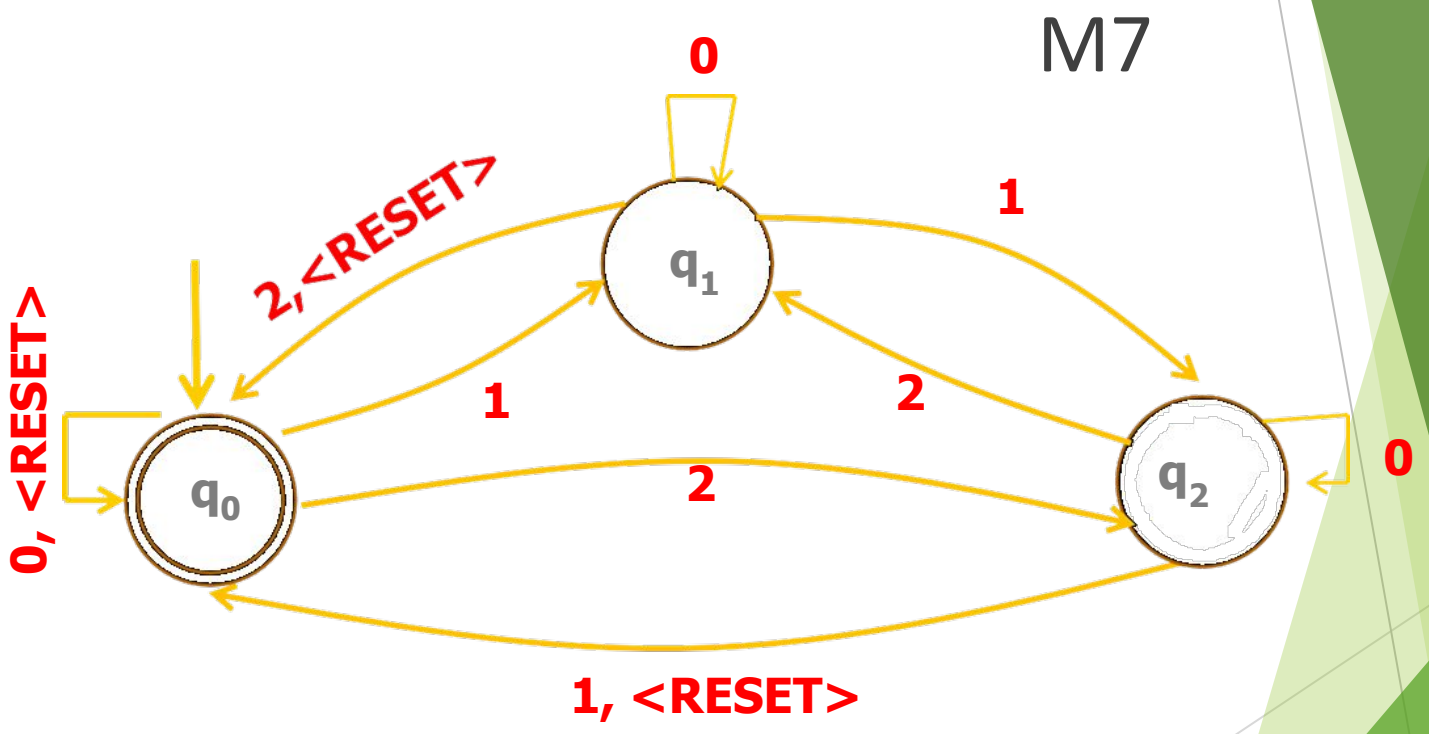
$L(M_5)=\{w \mid w$ começa e termina no mesmo símbolo$\}$

Autômatos Finitos



L(M6) = { w | w começa e termina com o mesmo símbolo}

Autômatos Finitos



$L(M_7)$={w | w a soma dos símbolos de w é múltiplo de 3 e o símbolo reset zera a contagem}

Autômatos Finitos (Definição formal)

- Situações em que não é possível escrever autômatos finitos por diagrama de estados:
  - Quando o diagrama é bem grande
  - Quando a descrição formal depende de algum parâmetro não especificado.

Autômatos Finitos (Definição formal)

- Generalização do problema múltiplo de 3:

Para cada $i \geq 1$ seja A_i, a linguagem de todas as cadeias em que a soma dos números é um múltiplo de i, exceto que a soma é reinicializada para 0 sempre que o símbolo <RESET> aparece.

Autômatos Finitos (Definição formal)

- Descrevemos uma máquina Bi, para reconhecer a linguagem Ai, formalmente da seguinte forma:
- Bi = {Σ, Qi, δ, q0, {q0}}
- Onde:
  - Σ = 0, 1,..., i-1
  - Qi = { q0,q1,..., q(i-1)}
  - δ é descrita como:
    - δ(qj, 0) = qj
    - δ(qj, n) = qk, onde k = j+n módulo i e n < i
    - δ(qj,<RESET>) = q0

Autômatos Finitos: Operações regulares

- Em aritmética, os objetos básicos são números e as ferramentas são operações para manipulá-los, tais como + e *.
- Na teoria da computação os objetos são linguagens e as ferramentas incluem operações especificamente projetadas para manipulá-las.
- Definimos três operações sobre linguagens, chamadas **operações regulares**, e as usamos para estudar propriedades de linguagens regulares.
- Isso ajuda a projetar autômatos para reconhecer linguagens específicas.
- Isso ajuda a provar que certas linguagens são não-regulares (isto é, além da capacidade de autômatos finitos).

Autômatos Finitos: Operações Regulares

- Sejam A e B linguagens. Definimos as operações regulares **união, concatenação, e estrela** da seguinte forma.

- União: $A \cup B = \{x \mid x \in A \text{ ou } x \in B\}$.
- Concatenação: $A \circ B = \{xy \mid x \in A \text{ e } y \in B\}$.
- Estrela: $A^* = \{x1x2 \ldots xk \mid k \geq 0 \text{ e cada } xi \in A\}$.

Operações regulares:

- Suponha que o alfabeto Σ seja o alfabeto padrão de 26 letras {a, b, ..., z}. Se A = {legal, ruim} e B = {garoto; garota}, então
  - $A \cup B$ = {legal, ruim, garoto, garota},
  - $A \circ B$ = {legalgaroto, legalgarota, ruimgaroto, ruimgarota}, e
  - A^* = {ε, legal, ruim, legallegal, legalruim, ruimlegal, ruimruim, legallegallegal; legallegalruim,legalruimlegal, legalruimruim,...}.

Operações Regulares

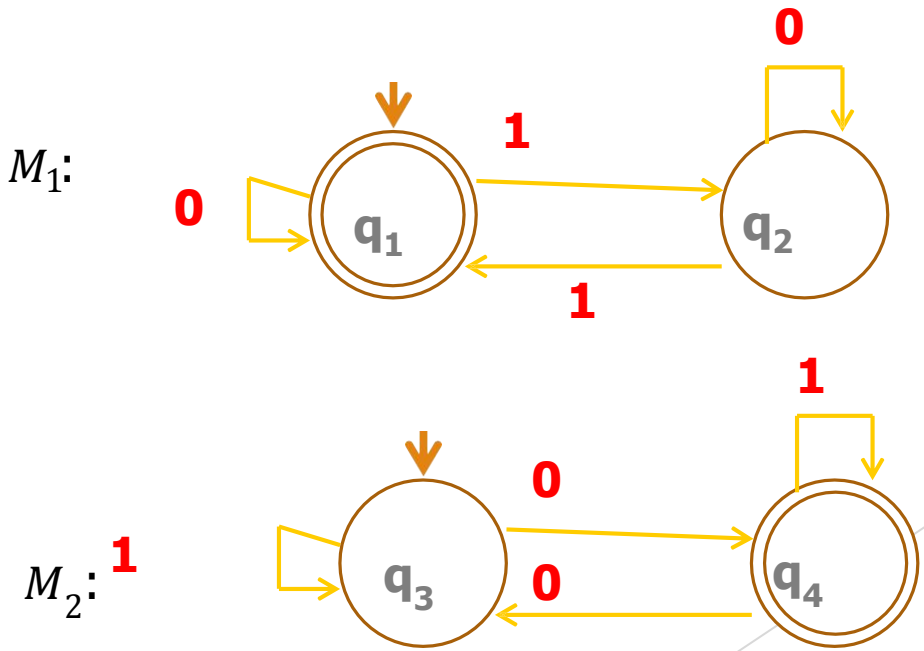
- Teorema: A classe de linguagens regulares é fechada sob a operação de união.
- Em outras palavras, se A1 e A2 são linguagens regulares, o mesmo acontece com A1 ∪ A2.
- Idéia da Prova: Se A1 e A2 são linguagens regulares, então existem AFs M1 e M2 que as reconhecem, respectivamente.
- Vamos fazer uma prova construtiva, ou seja, vamos construir um AF M, que reconheça A1 ∪ A2, a partir de M1e M2.

Operações Regulares

- Como vamos construir um AF M, que reconheça $A_1 \cup A_2$, a partir de M_1 e M_2?
- Simulando M_1 e M_2 simultaneamente.
- Para controlar ambas as simulações é preciso **guardar** o estado em que cada máquina estaria se ela tivesse lido até um ponto na entrada.
- Consequentemente, você precisa guardar um par de estados.
- Quantos pares de estados existem?

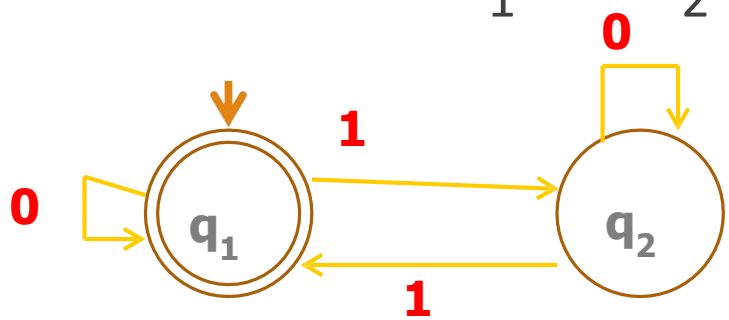
Operações Regulares

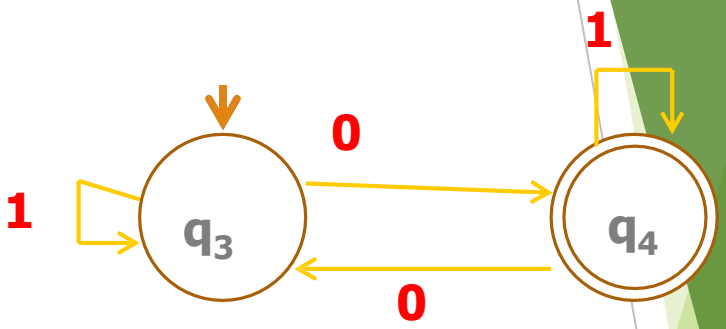
- Seja M_1 um AF que reconheça as cadeias de bits com um número par de 1s e M_2 reconhece aquelas com um número impar de zeros



Operações Regulares

Construindo $M_1 \cup M_2$



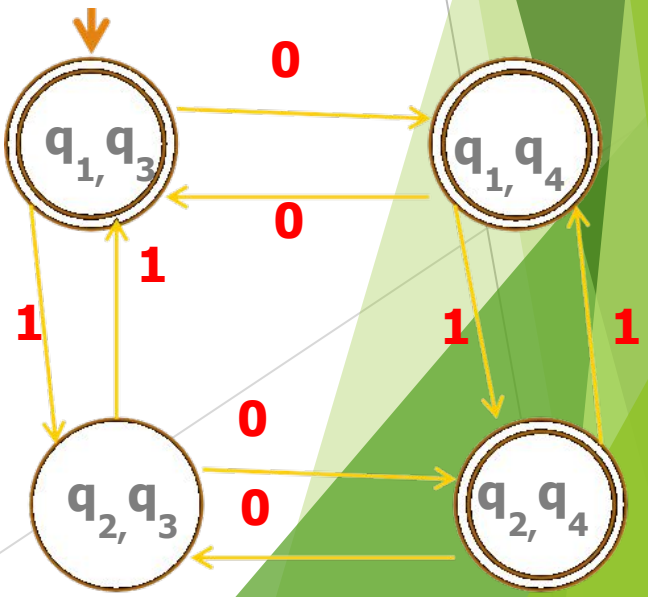


$M_1=(Q_1,\Sigma_1,\delta_1,q_1,F_1)$, $M_2=(Q_2,\Sigma_2,\delta_2,q_3,F_2)$

$M=(Q_1\times Q_2,\Sigma,\delta,(q_1,q_3),F)$

$\delta((r_1,r_2),a)=(\delta_1(r_1,a),\delta_2(r_2,a))$

$F_1=(F_1\times Q_2)\cup(Q_1\times F_2)$



Operações Regulares

- Construção formal de M para reconhecer $A_1 \cup A_2$, onde:

  $M = (\boldsymbol{Q}, \Sigma, \delta, \boldsymbol{q_0}, \mathbf{F})$.

1. $\mathbf{Q} = (r_1, r_2)$, onde $r_1 \in Q_1$ e $r_2 \in Q_2$
   - Escrito como produto cartesiano $Q_1 \times Q_2$
2. Σ é o mesmo em M_1 e M_2. Se o alfabeto forem diferentes então $\Sigma = \Sigma_1 \cup \Sigma_2$
3. $\delta((r_1, r_2), a) = \delta_1(r_1, a), \delta_2(r_2, a)$
4. $\mathbf{q_0}$ é o par (q_1, q_2)
   - Onde q_1 é o estado inicial de A_1 e q_2 é o estado inicial de A_2
5. $\mathbf{F} = \{(r_1, r_2)\}$, onde $r_1 \in F_1$ ou $r_2 \in F_2$
   - Equivalente à: $F = (F1 \times Q2 \cup Q1 \in F2)$
   - Obs: Não é equivalente à $F = Q1 \times Q2$. Por quê?

Exercício

Cada uma das linguagens a seguir é a união de duas linguagens mais simples. Em cada caso, construa AFDs para as linguagens mais simples, e depois combine-os usando a construção discutida para obter o diagrama de estados de um AFD para a linguagem dada.

a) {w|w tem pelo menos três **a**s ou pelo menos dois **b**s}
b) {w|w tem um número par de **a**s ou um ou dois **b**s}
c) {w|w tem um número ímpar de **a**s ou termina com um b}
d) {w|w tem comprimento par ou um número ímpar de **a**s}

Autômatos e Linguagens Não-Determinísticas

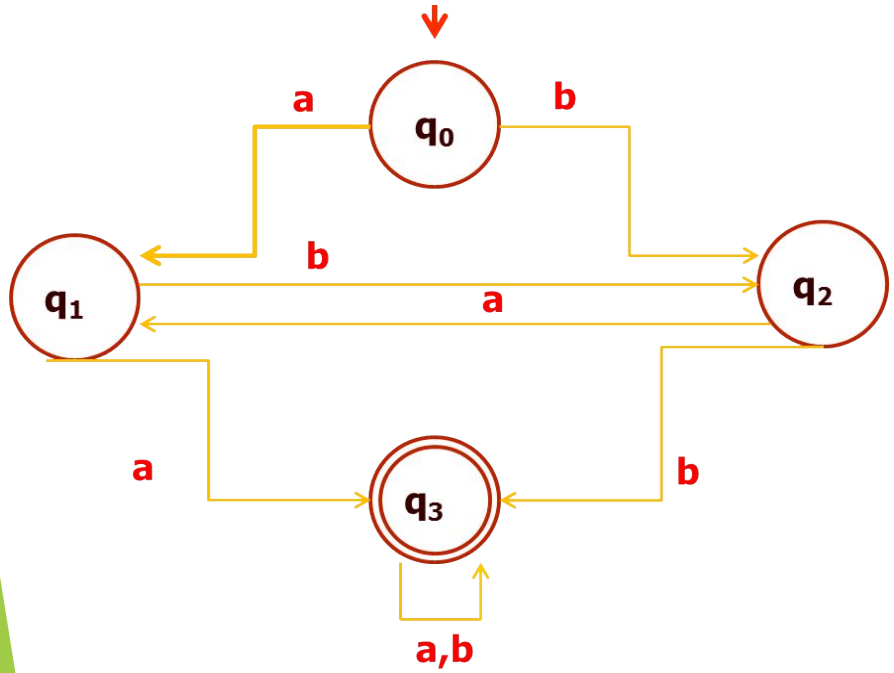
Determinismo × Não-Determinismo

- Os AFs que vimos até agora funcionam assim: quando a máquina está em um dado estado e lê o próximo símbolo de entrada, **sabemos qual será o próximo estado**, está determinado. Chamamos isso de computação *determinística*.

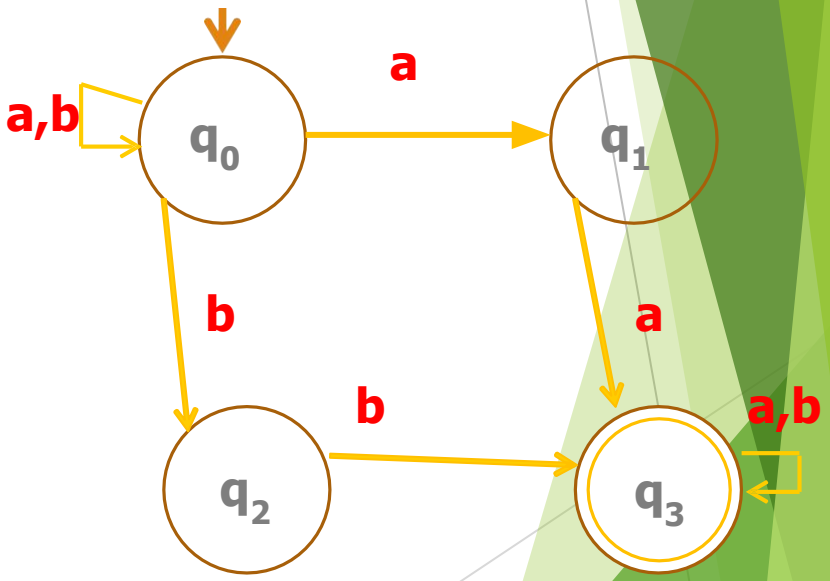
- Em um autômato finito *não-determinístico (AFN)*, **várias escolhas podem existir** para o próximo estado em qualquer ponto.

Autômatos finitos não-determinísticos

Considere um AFN N_1 que aceita as cadeias sobre o alfabeto {a,b} que possuem aa ou bb como subcadeia.



Todo estado de um AFD sempre tem exatamente uma seta de transição saindo para cada símbolo no alfabeto



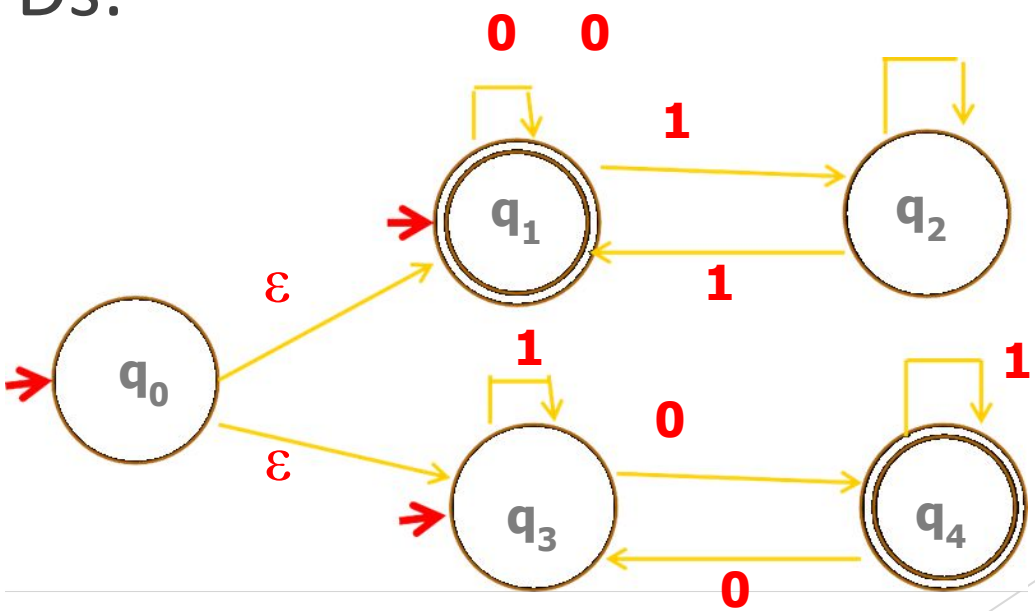
Em um AFN um estado pode ter zero, uma ou muitas setas saindo para cada símbolo do alfabeto

Determinismo × Não-Determinismo

- Algumas diferenças práticas
  - Em um AFN um estado pode ter zero, uma ou várias setas saindo para cada símbolo do alfabeto.
  - Em geral, um AFN pode ter setas rotuladas com membros do alfabeto ou com ϵ
    - Zero uma ou mais setas podem sair de cada estado com o rótulo ϵ

Autômatos finitos não-determinísticos

Vamos construir um AFN que aceita cadeias com um número par de 1s ou ímpar de 0s a partir desses AFDs:

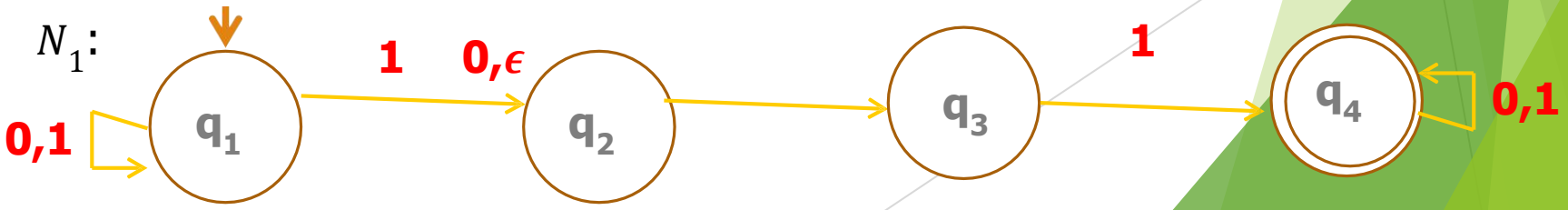


Em geral, um AFN pode ter
ter setas rotuladas com membros do alfabeto
ou com ε.

Autômatos finitos não-determinísticos

Como devemos pensar no processamento de um AFN?

- No estado q_1: Se receber '1', N_1 se divide em múltiplas cópias de si mesma e segue *todas* as possibilidades em **paralelo**
- Se existirem escolhas subsequentes, a máquina divide-se novamente
- Se o próximo símbolo de entrada não aparece em qualquer das setas, a cópia da máquina morre.
- Um estado com uma seta ϵ: sem ler qualquer entrada, a máquina divide-se em múltiplas cópias, uma seguindo cada uma das setas que tem ϵ e uma permanecendo no estado corrente.
- Finalmente, se *qualquer uma* dessas cópias da máquina está em um estado de aceitação no final da entrada, o AFN aceita a cadeia de entrada



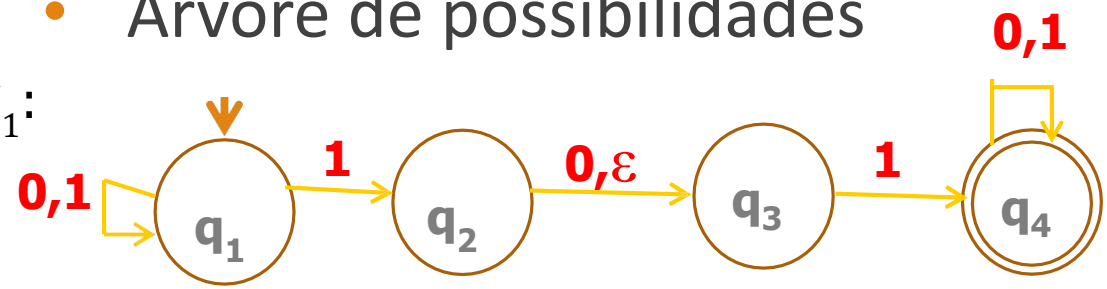
A computação em um AFN

- Não-determinismo pode ser visto como uma espécie de **computação paralela** na qual múltiplos e independentes “processos” ou “threads” podem estar rodando concorrentemente.
- Se pelo menos um desses processos aceita então a computação inteira aceita

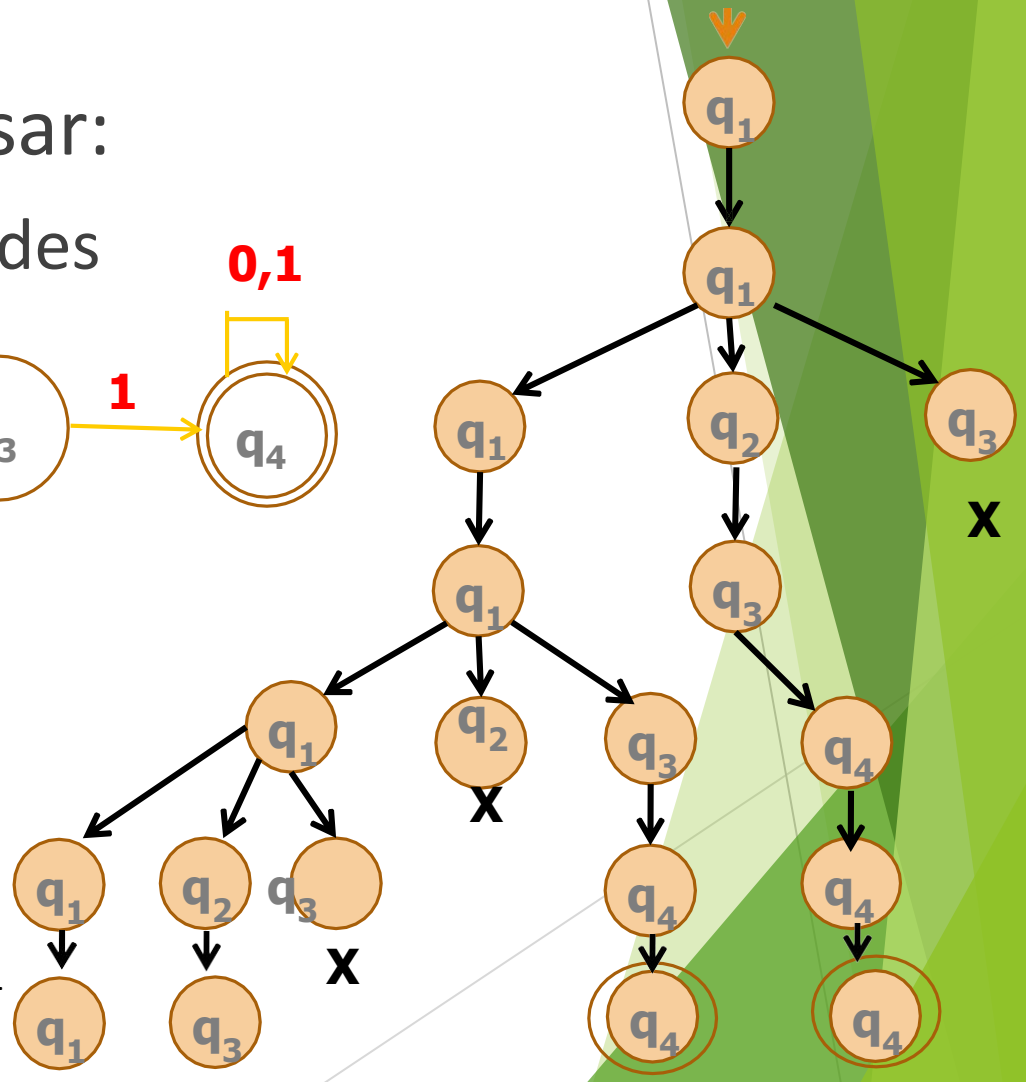
A computação em um AFN

- Outra forma de pensar:
  - Árvore de possibilidades

N_1:



Entrada **010110**



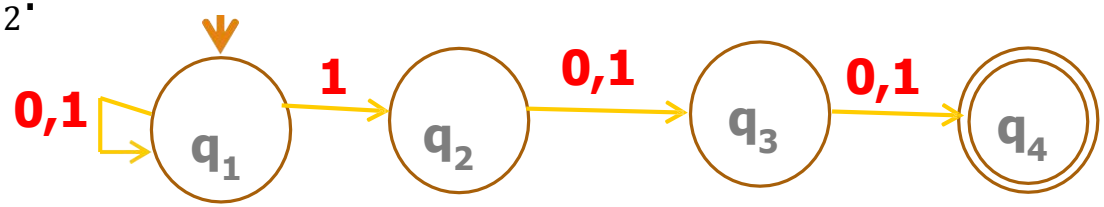
Qual linguagem N_1 reconhece?

N_1 aceita todas as cadeias que contêm 101 ou 11 como uma subcadeia

Exemplo 1.30

Seja A a linguagem consistindo de todas as cadeias sobre {0,1} contendo um 1 na terceira posição a partir do final (por exemplo, 000100 está em A, mas 0011 não).

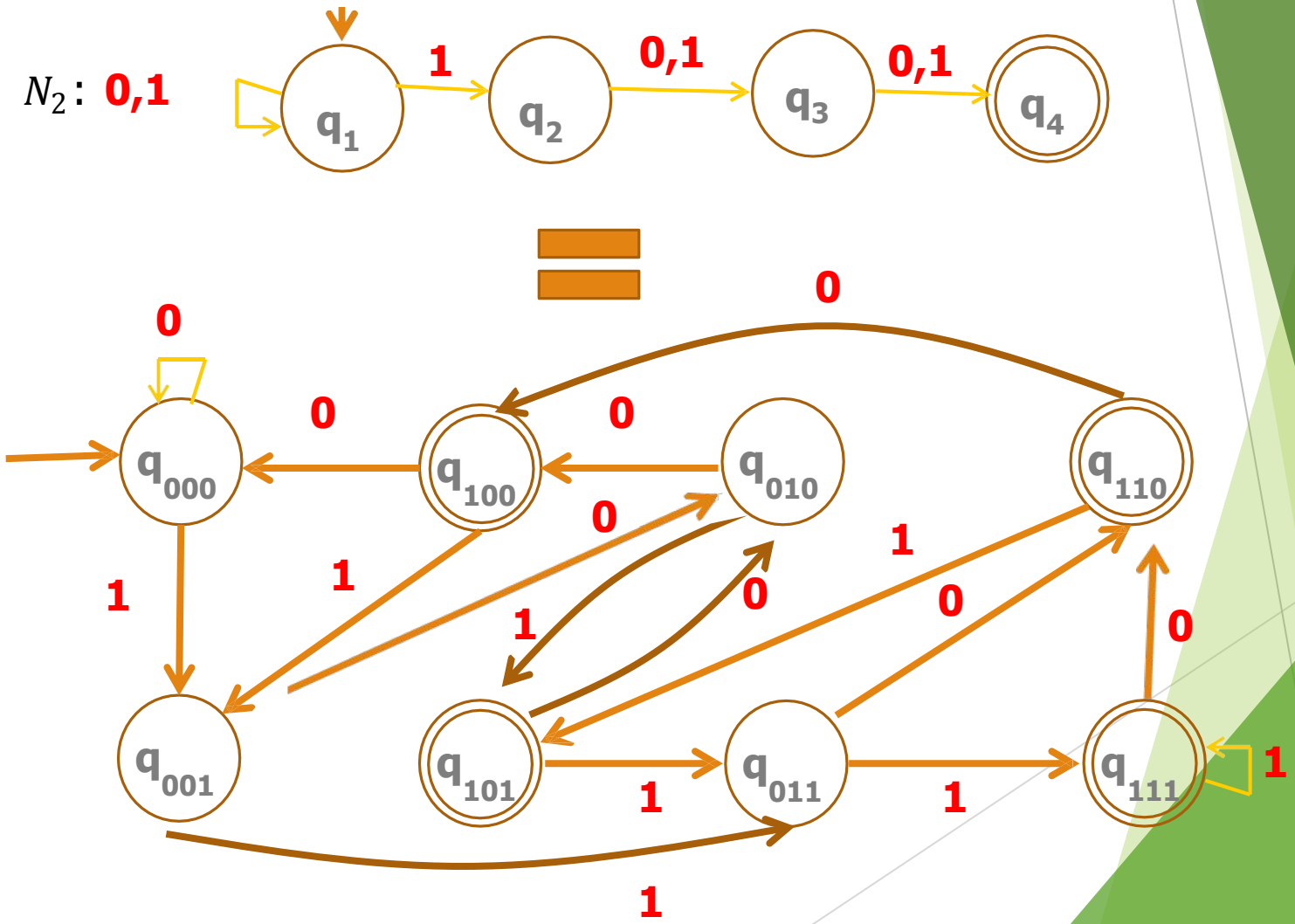
N_2:



Utilidade dos AFNs

- Construir AFNs é muitas vezes mais fácil que construir diretamente AFDs
  - Um AFN pode ser muito menor que sua contrapartida AFD
  - Ou seu funcionamento pode ser mais fácil de entender
- Todo AFN pode ser convertido em um AFD equivalente (veremos adiante).

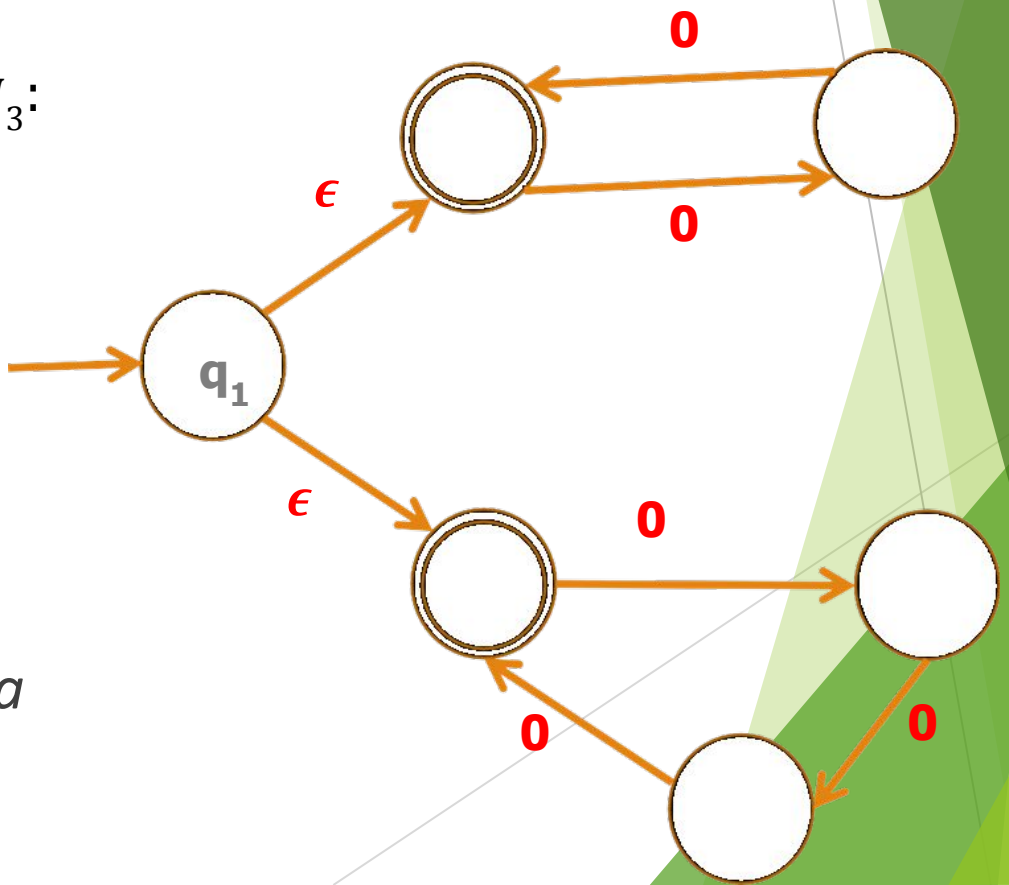
Utilidade dos AFNs



Exemplo 1.33

Considere o seguinte AFN N_3 que tem o alfabeto de entrada {0} consistindo de um único símbolo.

N_3:



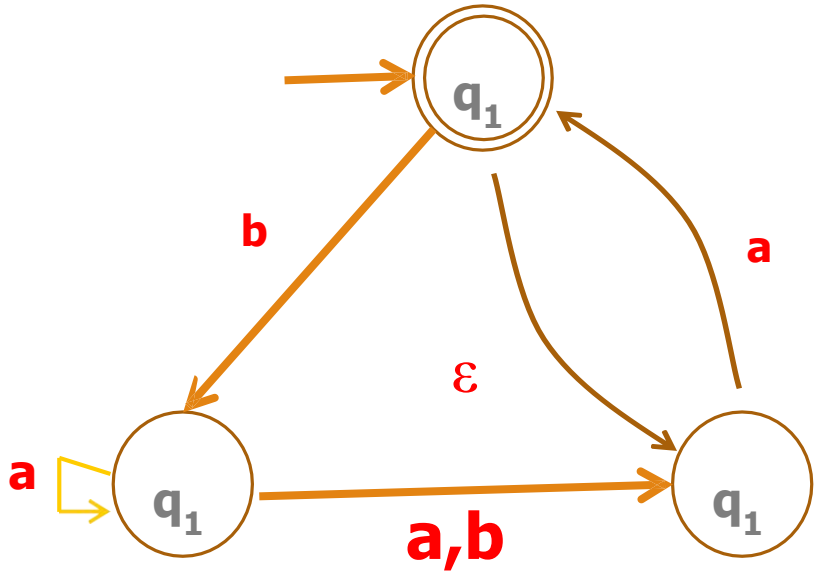
Verifica-se a conveniência de ser ter seta ϵ

Exemplo 1.35

Esse autômato aceita as cadeias ε, a, baba e baa?

E as cadeias b, bb, babba?

N_4



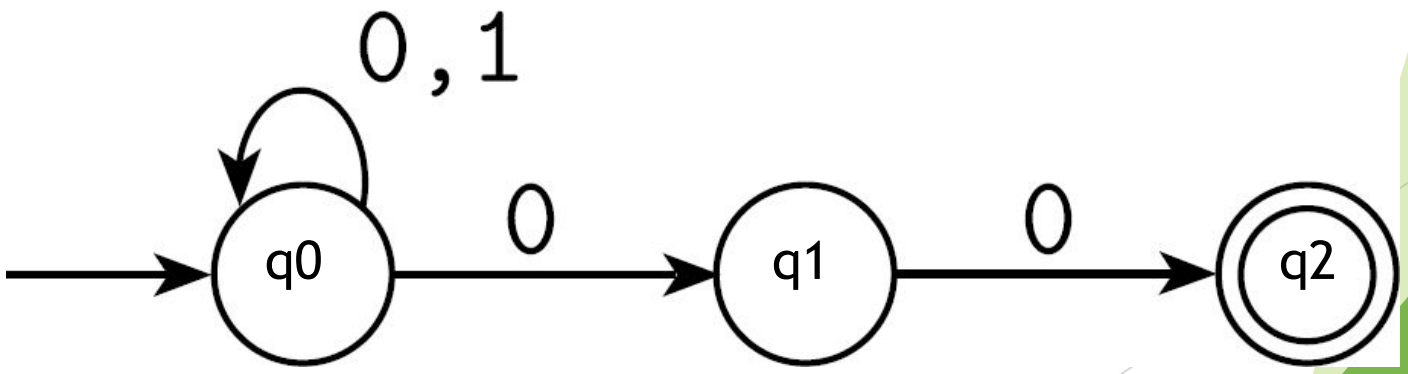
Exercícios em sala

Dê o diagrama de estados de AFNs que reconheça a linguagem {w|w termina em 00} com três estados

Exercícios em sala

Dê o diagrama de estados de AFNs que reconheça a linguagem {w|w termina em 00} com três estados

Resposta:



Exercícios em sala

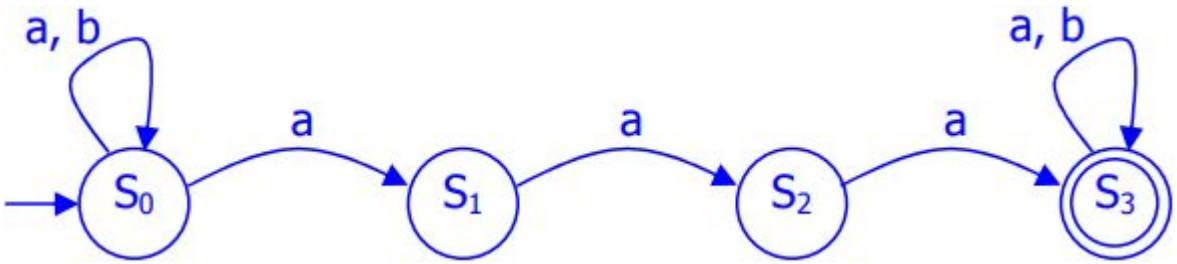
Desenvolva autômatos que reconheçam as seguintes linguagens. $\Sigma = \{a,b\}$

$\{w \mid aaa$ é subpalavra de $w\}$

Exercícios em sala

Desenvolva autômatos que reconheçam as seguintes linguagens. $\Sigma = \{a,b\}$

$\{w \mid aaa$ é subpalavra de $w\}$



Exercícios em sala

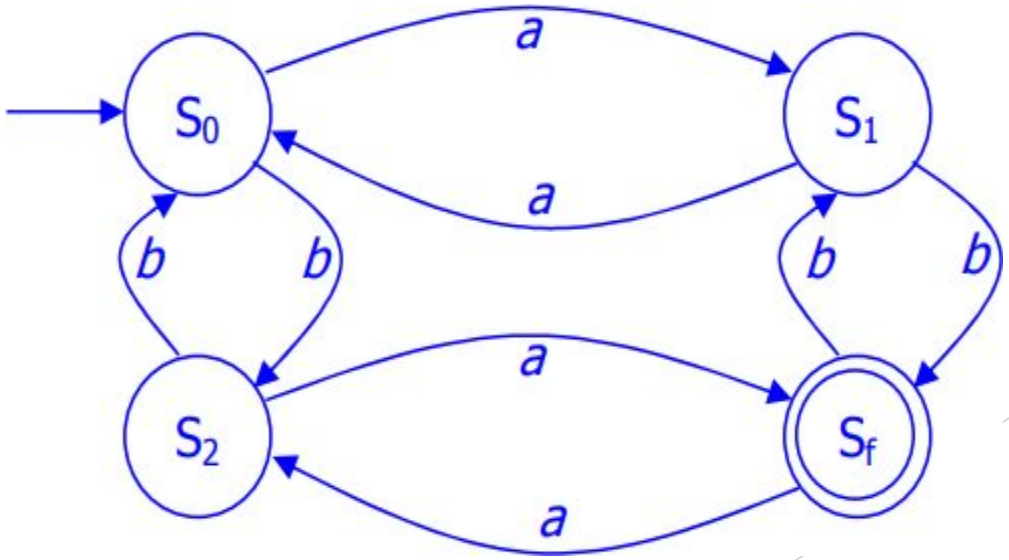
Desenvolva autômatos que reconheçam as seguintes linguagens. $\Sigma = \{a,b\}$

$\{w \mid w$ possui uma quantidade ímpar de a e de b$\}$

Exercícios em sala

Desenvolva autômatos que reconheçam as seguintes linguagens. $\Sigma = \{a,b\}$

{w | w possui uma quantidade ímpar de a e de b}



Exercícios em sala

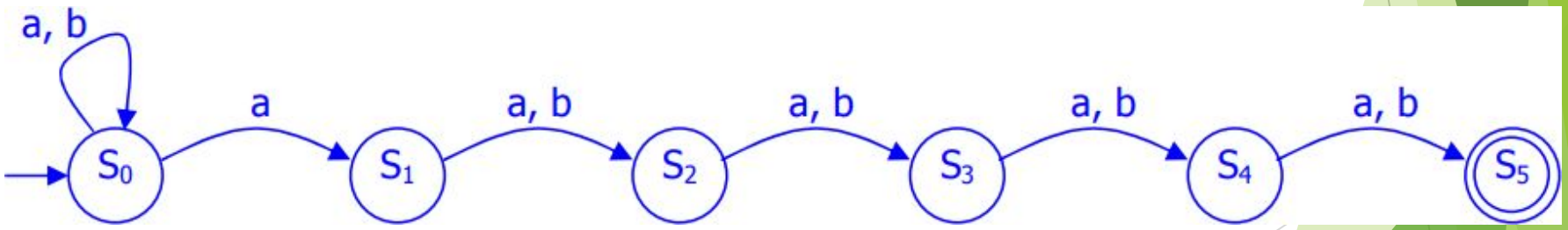
- Desenvolva autômatos que reconheçam as seguintes linguagens. $\Sigma = \{a,b\}$

{w | o quinto símbolo da direita para a esquerda de w é a}

Exercícios em sala

- Desenvolva autômatos que reconheçam as seguintes linguagens. $\Sigma = \{a,b\}$

{w | o quinto símbolo da direita para a esquerda de w é a}



AFN: Descrição Formal

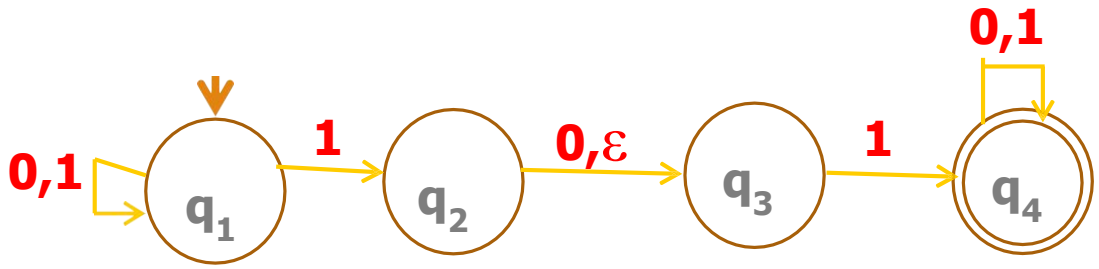
- Em um AFD a função de transição toma um estado e um símbolo de entrada e produz o próximo estado.
- Em um AFN a função de transição toma um estado e um símbolo de entrada *ou a cadeia vazia* e produz *o* ***conjunto de próximos estados possíveis***.
- Desse modo, vamos usar a notação $\Sigma_\varepsilon = \Sigma \cup \varepsilon$.
- E definir a função de transição da seguinte forma: $\delta: Q \times \Sigma_\varepsilon \rightarrow P(Q)$.

AFN: Definição Formal

Um *autômato finito não-determinístico* é uma 5-upla $(Q, \Sigma, \delta, q_0, F)$, onde

1. Q é um conjunto finito denominado os *estados*,
2. Σ é um conjunto finito denominado *alfabeto*,
3. $\delta: Q \times \Sigma_\varepsilon \rightarrow P(Q)$ é a *função de transição,* $P(Q)$ conjunto das partes de Q.
4. $q_0 \in Q$ é o *estado inicial*, e
5. $F \subseteq Q$ é o *conjunto de estados de aceitação (ou finais).*

Definição formal: exemplo



$Q=\{q_1,q_2,q_3,q_4\}$

$\Sigma=\{0,1\}$

q_1 é o estado inicial

$F=\{q_4\}$

δ	0	1	ε
q_1	$\{q_1\}$	$\{q_1,q_2\}$	$\varnothing$
q_2	$\{q_3\}$	$\varnothing$	$\{q_3\}$
q_3	$\varnothing$	$\{q_4\}$	$\varnothing$
q_4	$\{q_4\}$	$\{q_4\}$	$\varnothing$

Exercício do livro

1.7) Dê diagramas de estado de AFNs com o número especificado de estados reconhecendo cada uma das linguagens a seguir. Em todos os casos o alfabeto é {0,1}.

b.{w|w contém a subcadeia 0101, isto é, w = x0101y para algum x e algum y}

c.{w | w contém um número par de 0s, ou contém exatamente dois 1s}

d. A linguagem {0} com dois estados

Equivalência entre AFNs e AFDs

Equivalência entre AFNs e AFDs

- Autômatos finitos determinísticos e não-determinísticos **reconhecem a mesma classe de linguagens.**
  - É útil porque descrever um AFN é mais fácil
- **Teorema: Todo autômato finito não-determinístico tem um autômato finito determinístico equivalente.**
- Uma **linguagem é regular** se e somente se algum **autômato finito não-determinístico a reconhece**
  - **Por consequência essa linguagem também possui um AFD que a reconhece**

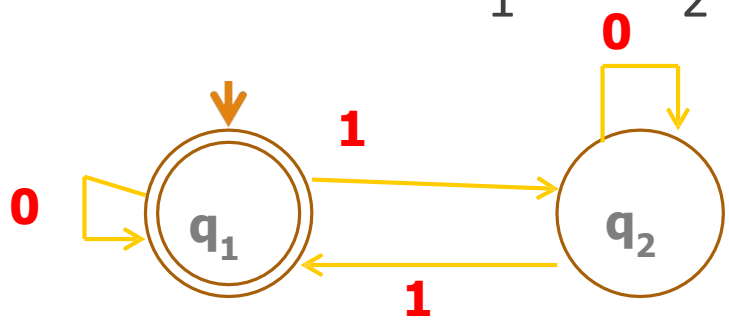
AFN: Operações regulares

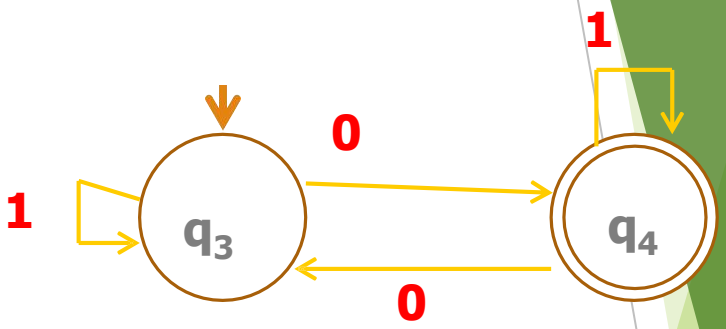
Operações Regulares

- **Teorema:** A classe de linguagens regulares é fechada sob a operação de **união**.
  - Em outras palavras, se A1 e A2 são linguagens regulares, o mesmo acontece com A1 ∪ A2.

Operações Regulares

Construindo $M_1 \cup M_2$



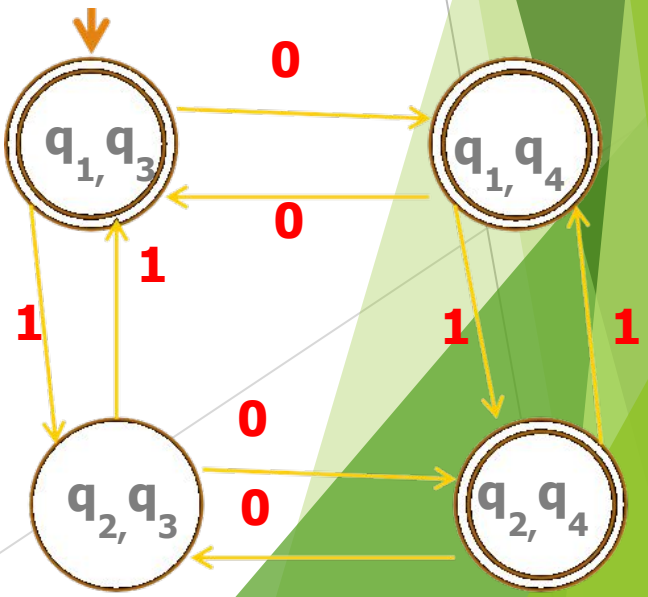


$M_1=(Q_1,\Sigma_1,\delta_1,q_1,F_1)$, $M_2=(Q_2,\Sigma_2,\delta_2,q_3,F_2)$

$M=(Q_1\times Q_2,\Sigma,\delta,(q_1,q_3),F)$

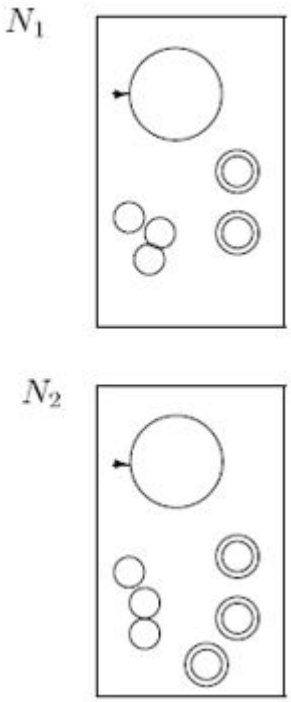
$\delta((r_1,r_2),a)=(\delta_1(r_1,a),\delta_2(r_2,a))$

$F_1=(F_1\times Q_2)\cup(Q_1\times F_2)$



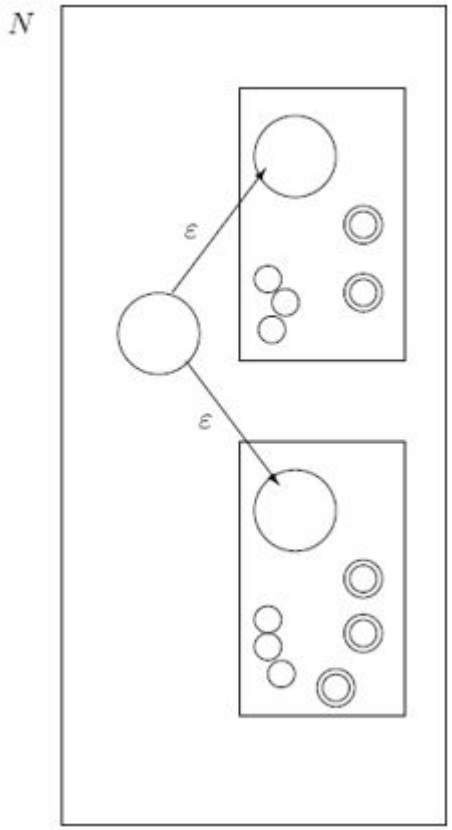
União

$N_1 = (Q_1, \Sigma_1, \delta_1, q_1, F_1)$
reconhece A_1



$N_2 = (Q_2, \Sigma_2, \delta_2, q_2, F_2)$
reconhece A_2.

$N = (Q, \Sigma, \delta, q_0, F)$
reconhece $A_1 \cup A_2$



$Q = Q_1 \cup Q_2 \cup \{q_0\}$

$F = F_1 \cup F_2$

$\delta(q, \Sigma) =$

$\delta_1(q,a)$ se $q \in Q_1$,

$\delta_2(q,a)$ se $q \in Q_2$,

$\{q_1, q_2\}$ se $q = q_0$ e $a = \varepsilon$

$\varnothing$ se $q = q_0$ e $a \neq \varepsilon$

Operações Regulares

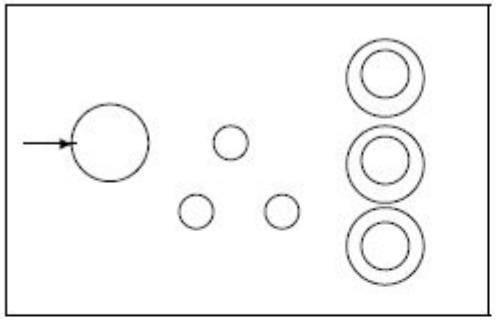
- **Teorema**: A classe de linguagens regulares é fechada sob a operação de **concatenação**.
  - Se A_1 e A_2 são linguagens regulares, então existem AFNs N_1 e N_2 que as reconhecem, respectivamente.
  - Então um AFN N, que reconheça $A_1 \circ A_2$, a partir de N_1e N_2 também é uma linguagem regular

Concatenação

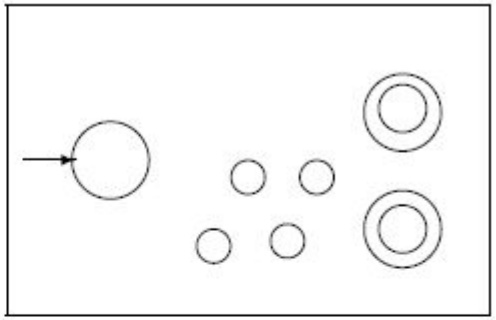
$N_1 = (Q_1, \Sigma_1, \delta_1, q_1, F_1)$ rec. A_1

$N_2 = (Q_2, \Sigma_2, \delta_2, q_2, F_2)$ rec. A_2.

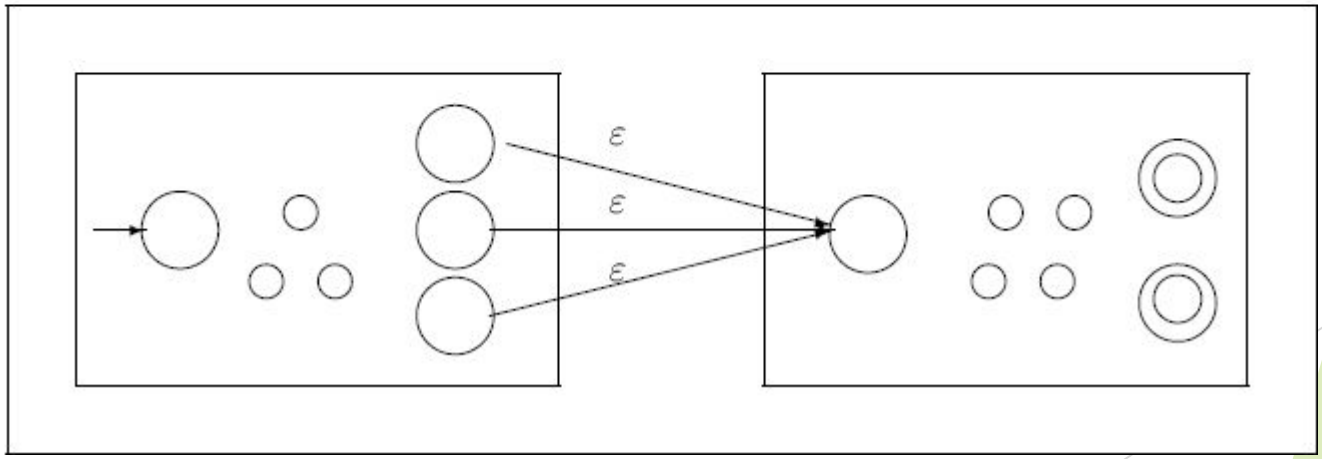
N_1



N_2



N $N = (Q, \Sigma, \delta, q_1, F_2)$ reconhece $A_1 \circ A_2$ $Q = Q_1 \cup Q_2$



$\delta(q,a) = \delta_1(q,a)$ se $q \in Q_1$ e $q \notin F_1$

$\delta_1(q,a) \cup \{q_2\}$ se $q \in F_1$ e $a = \varepsilon$

$\delta_1(q,a)$ se $q \in F_1$ e $a \neq \varepsilon$

$\delta_2(q,a)$ se $q \in Q_2$

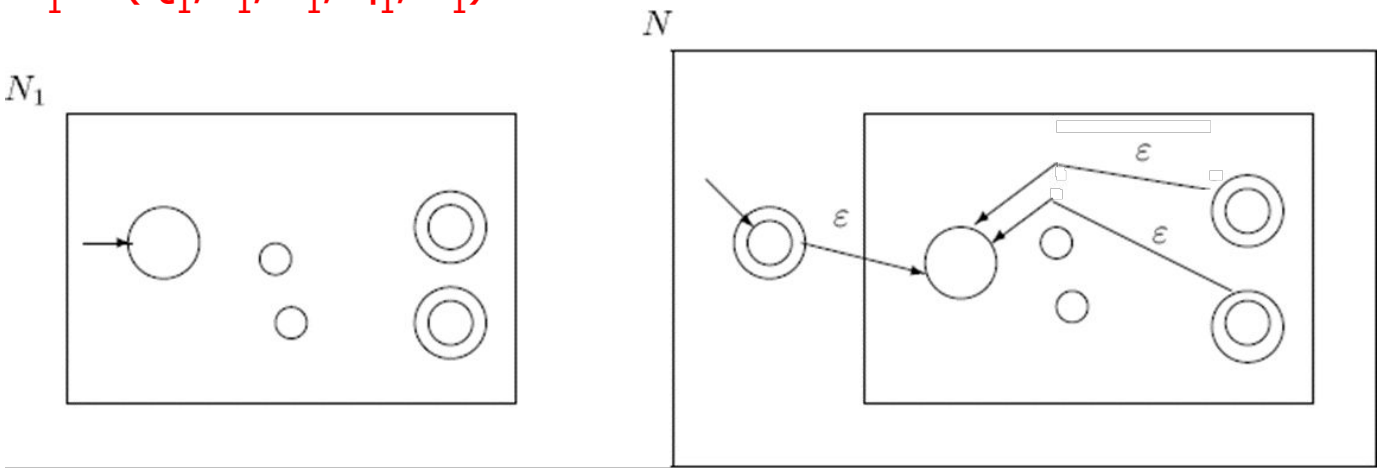
Operações Regulares

- Teorema: A classe de linguagens regulares é fechada sob a operação estrela.
- Se A1 é uma linguagem regular, então existe um AFN N1 que a reconheça.
- Vamos construir um AFN N, que reconheça A1*, a partir de N1.

Estrela

$N = (Q, \Sigma, \delta, q_0, F)$ reconhece A_1^*

$N_1 = (Q_1, \Sigma_1, \delta_1, q_1, F_1)$



$Q = Q_1 \cup \{q_0\}$ $\quad F = F_1 \cup \{q_0\}$

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & \text{se } q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q,a) & \text{se } q \in F_1 \text{ e } a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_1\} & \text{se } q \in F_1 \text{ e } a = \varepsilon \\ \{q_1\} & \text{se } q = q_0 \text{ e } a = \varepsilon \\ \varnothing & \text{se } q = q_0 \text{ e } a \neq \varepsilon \end{cases}$$

Expressões Regulares

Expressões Regulares

- Na aritmética, podemos usar as operações + e $\times$ para montar expressões tais como $(5 + 3) \times 4$.
- Similarmente, podemos usar as operações regulares para montar expressões descrevendo linguagens, que são chamadas ***expressões regulares***.
- Por exemplo: $(0 \cup 1)^{\circ}0^*$
  - O símbolo da concatenação pode ser suprimido: $(0 \cup 1)0^*$
- O valor de uma expressão aritmética é um número. No exemplo é 32.
- O valor de uma expressão regular é uma **linguagem**.
- No exemplo: a linguagem das cadeias binárias que começam com 1 ou 0 seguido por um número qualquer de 0s.

Expressões Regulares

Outro exemplo: $(0 \cup 1)^*$

Ela começa com a linguagem $(0 \cup 1)$ e aplica a operação *.

O valor dessa expressão é a linguagem consistindo de todas as possíveis cadeias de 0s e 1s.

Se $\Sigma = \{0,1\}$, podemos escrever Σ como abreviação para a expressão regular $(0 \cup 1)$.

Expressões Regulares

Genericamente, se Σ for um alfabeto qualquer, a expressão regular Σ descreve a linguagem consistindo de todas as cadeias de comprimento 1 sobre esse alfabeto.

Σ^* descreve a linguagem consistindo de todas as cadeias sobre esse alfabeto.

Expressões Regulares

Qual é a linguagem Σ^*1 ?

Expressões Regulares

Qual é a linguagem Σ^*1 ?

É a linguagem que contém todas as cadeias que terminam em 1.

Expressões Regulares

E a linguagem $(0\ \Sigma^*) \cup (\Sigma^*\ 1)$?

Expressões Regulares

E a linguagem $(0\ \Sigma^*) \cup (\Sigma^*\ 1)$?

Consiste de todas as cadeias que começam com 0 ou terminam com 1.

Precedência dos operadores

- A operação estrela é feita primeiro, seguida por concatenação, e finalmente união, a menos que parênteses sejam usados para mudar a ordem usual.

1. ()
2. *
3. ∘
4. ∪

Expressões Regulares: Definição Formal

- Dizemos que R é uma *expressão regular* se R for:

1. a para algum a no alfabeto Σ,
2. ε,
3. $\varnothing$,
4. $(R_1 \cup R_2)$, onde R_1 e R_2 são expressões regulares,
5. $(R_1 \circ R_2)$, onde R_1 e R_2 são expressões regulares, ou
6. (R_1^*), onde R_1 é uma expressão regular.
7. Nos itens 1 e 2, as expressões regulares a e ε representam as linguagens {a} e $\{\varepsilon\}$, respectivamente.

Expressões Regulares: Notação

$R^+ = RR^*$

$R^* = R^+ \cup \{\varepsilon\}$

R^K : concatenação de k's Rs.

Quando queremos distinguir entre uma expressão regular R e a linguagem que ela descreve, escrevemos L(R) como sendo a linguagem de R.

Expressões Regulares: Exemplos onde $\Sigma=\{0,1\}$

- 0^*10^* = {w | w contem um único 1}.
- $\Sigma^*1\Sigma^*$ = {w | w tem pelo menos um símbolo 1}.
- $1^*(01^+)^*$ = {w | todo 0 em w é seguido por pelo menos um 1}.
- $(\Sigma\Sigma)^*$ = {w | w é uma cadeia de comprimento par}.
- $01 \cup 10$ = {01, 10}
- $0\Sigma^*0 \cup 1\Sigma^*1 \cup 0 \cup 1$ = {w | w começa e termina com o mesmo símbolo}.

Expressões Regulares: Exemplos onde $\Sigma=\{0,1\}$

- $(0 \cup \varepsilon)1^* = 01^* \cup 1^*$
- $(0 \cup \varepsilon)(1 \cup \varepsilon) = \{\varepsilon,0,1,01\}$.
- $1^*\varnothing = \varnothing$
  - Concatenar um conjunto vazio a qualquer conjunto produz conjunto vazio
- $\varnothing^* = \{\varepsilon\}$.
  - A operação estrela junta qualquer número (inclusive 0) de cadeias da linguagem para obter uma cadeia no resultado. Por isso a cadeia vazia ε é sempre um membro da operação estrela. Se a linguagem for vazia, a operação estrela resulta apenas na cadeia vazia.

Expressões Regulares: Identidades

- $R \cup \emptyset = R$
- $R \circ \varepsilon = R$
- $R \cup \{\varepsilon\}$ = pode ser diferente de R
- $R \circ \emptyset =$ É sempre $\emptyset$

Expressões Regulares

Expressões regulares são ferramentas úteis no desenho de compiladores para linguagens de programação.

Objetos elementares, *tokens*, tais como os nomes de variáveis e constantes podem ser descritos com expressões regulares.

A partir da sintaxe dos tokens, sistemas automáticos podem gerar analisadores léxicos, a parte do compilador que inicialmente processa o programa de entrada.

Expressões Regulares: Exemplo

Uma constante numérica que pode incluir uma parte fracionária e/ou um sinal pode ser descrita como um membro da linguagem:

$(+ \cup - \cup \varepsilon) (D^{+} \cup D^{+}.D^{*} \cup D^{*}.D^{+})$ Onde

$D=\{0,1,2,3,4,5,6,7,8,9\}$

Cadeias geradas: 72, 3.14159, +7. e -.01